

Name:Manav Mangesh Uttekar

Roll no.: 71

Problem statement: A Dictionary stores keywords and its meanings. Provide facility for adding new keywords, deleting keywords, updating values of any entry. Provide facility to display whole data sorted in ascending/ Descending order. Also find how many maximum comparisons may require for finding any keyword. Use Height balance tree and find the complexity for finding a keyword

class Node:

```
def __init__(self, key, meaning):
    self.key = key
    self.meaning = meaning
    self.left = None
    self.right = None
    self.height = 1
```

class AVLTreeDict:

```
def insert(self, root, key, meaning):
    if not root:
        return Node(key, meaning)
    if key < root.key:
        root.left = self.insert(root.left, key, meaning)
    elif key > root.key:
        root.right = self.insert(root.right, key, meaning)
    else:
        print("Key already exists.")
        return root

    root.height = 1 + max(self.getHeight(root.left),
self.getHeight(root.right))
    balance = self.getBalance(root)

    # Balancing
    if balance > 1 and key < root.left.key:
        return self.rightRotate(root)
    if balance < -1 and key > root.right.key:
        return self.leftRotate(root)
    if balance > 1 and key > root.left.key:
        root.left = self.leftRotate(root.left)
        return self.rightRotate(root)
    if balance < -1 and key < root.right.key:
        root.right = self.rightRotate(root.right)
        return self.leftRotate(root)

    return root

def delete(self, root, key):
    if not root:
        return root
    if key < root.key:
        root.left = self.delete(root.left, key)
    elif key > root.key:
        root.right = self.delete(root.right, key)
```

```

        else:
            if not root.left: return root.right
            elif not root.right: return root.left
            temp = self.getMinValueNode(root.right)
            root.key, root.meaning = temp.key, temp.meaning
            root.right = self.delete(root.right, temp.key)

        root.height = 1 + max(self.getHeight(root.left),
self.getHeight(root.right))
        balance = self.getBalance(root)

        # Rebalance
        if balance > 1 and self.getBalance(root.left) >= 0:
            return self.rightRotate(root)
        if balance < -1 and self.getBalance(root.right) <= 0:
            return self.leftRotate(root)
        if balance > 1 and self.getBalance(root.left) < 0:
            root.left = self.leftRotate(root.left)
            return self.rightRotate(root)
        if balance < -1 and self.getBalance(root.right) > 0:
            root.right = self.rightRotate(root.right)
            return self.leftRotate(root)
        return root

def update(self, root, key, new_meaning):
    node, cmp = self.search(root, key)
    if node:
        node.meaning = new_meaning
        print(f"Updated '{key}' with new meaning.")
    else:
        print("Key not found.")

def search(self, root, key):
    count = 0
    while root:
        count += 1
        if key == root.key:
            print(f"Found '{key}' in {count} comparisons.")
            return root, count
        elif key < root.key:
            root = root.left
        else:
            root = root.right
    print("Key not found.")
    return None, count

def inOrder(self, root):
    if root:
        self.inOrder(root.left)
        print(f"{root.key}: {root.meaning}")
        self.inOrder(root.right)

def reverseOrder(self, root):
    if root:

```

```

        self.reverseOrder(root.right)
        print(f"{root.key}: {root.meaning}")
        self.reverseOrder(root.left)

    def getHeight(self, root):
        return root.height if root else 0

    def getBalance(self, root):
        return self.getHeight(root.left) - self.getHeight(root.right) if
root else 0

    def leftRotate(self, z):
        y = z.right
        T2 = y.left
        y.left = z
        z.right = T2
        z.height = 1 + max(self.getHeight(z.left),
self.getHeight(z.right))
        y.height = 1 + max(self.getHeight(y.left),
self.getHeight(y.right))
        return y

    def rightRotate(self, y):
        x = y.left
        T2 = x.right
        x.right = y
        y.left = T2
        y.height = 1 + max(self.getHeight(y.left),
self.getHeight(y.right))
        x.height = 1 + max(self.getHeight(x.left),
self.getHeight(x.right))
        return x

    def getMinValueNode(self, node):
        while node.left:
            node = node.left
        return node

# ---- Demo ----
if __name__ == "__main__":
    tree = AVLTreeDict()
    root = None

    while True:
        print("\n1. Add 2. Delete 3. Update 4. Display Asc 5. Desc 6.
Search 7. Exit")
        ch = int(input("Enter choice: "))
        if ch == 1:
            k = input("Enter keyword: ")
            m = input("Enter meaning: ")
            root = tree.insert(root, k, m)
        elif ch == 2:
            k = input("Enter keyword to delete: ")
            root = tree.delete(root, k)

```

```
elif ch == 3:
    k = input("Enter keyword to update: ")
    m = input("Enter new meaning: ")
    tree.update(root, k, m)
elif ch == 4:
    print("Ascending order:")
    tree.inOrder(root)
elif ch == 5:
    print("Descending order:")
    tree.reverseOrder(root)
elif ch == 6:
    k = input("Enter keyword to search: ")
    _, cmp = tree.search(root, k)
else:
    break
```

Output:

```
Enter keyword: Apple
Enter meaning: A fruit

Enter choice: 1
Enter keyword: Banana
Enter meaning: Another fruit

Enter choice: 1
Enter keyword: Cat
Enter meaning: An animal

Enter choice: 4
Ascending order:
Apple: A fruit
Banana: Another fruit
Cat: An animal

Enter choice: 5
Descending order:
Cat: An animal
Banana: Another fruit
Apple: A fruit

Enter choice: 6
Enter keyword to search: Banana
Found 'Banana' in 2 comparisons.

Enter choice: 3
Enter keyword to update: Banana
Enter new meaning: A yellow fruit
Updated 'Banana' with new meaning.

Enter choice: 4
Ascending order:
Apple: A fruit
Banana: A yellow fruit
Cat: An animal
```